Code-Layout, Readability and Reusability

Date: 29 September 2026

Team ID: SWTID-2026-3268

Project Name: Implement Client Script & UI Policy (Incident)

Maximum Marks: 5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Consistent formatting is maintained throughout the Client Script and UI Policy configuration.
2	Proper File Structure	Files and folders are logically organized	Yes	Incident-related configuration components are organized according to their purpose.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables and field references use clear and meaningful names.
4	Function / Method Names	Functions are descriptively named	Yes	Functions and handlers are named to describe their intended purpose.
5	Code Comments	Inline and block comments explain logic	Yes	Comments explain important client-side validation and form behavior.
6	Modular Design	Code is split into reusable functions/modules	Yes	Client Script and UI Policy logic are separated based on their responsibilities.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Duplicate and unnecessary logic is avoided in the Incident implementation.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Client-side validation helps prevent incorrect or incomplete Incident form entries.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Incident Client Script	JavaScript / ServiceNow	Incident form client-side behavior and validation	High
2	Incident UI Policy	ServiceNow UI Policy	Incident form field visibility and mandatory settings	High
3	Client-side Field Validation	JavaScript / ServiceNow	Incident form data validation before submission	High
4	Field Visibility / Mandatory Rules	ServiceNow UI Policy	Controlling Incident field visibility and mandatory state	High
5	Incident Form Configuration	ServiceNow Platform	Incident management form and workflow configuration	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	The Client Script and UI Policy configuration is clearly organized by function and responsibility.
Readability	5	The configuration uses clear naming, structured logic, and understandable field-level rules.
Reusability	4	The client-side rules and UI Policy patterns can be reused for similar Incident form requirements.
Documentation / Comments	4	Important validation and form-behavior logic is explained with appropriate comments.
Overall Score	18 / 20	The implementation demonstrates good structure, readability, documentation, and reuse potential.

Project Summary: This project implements Client Script and UI Policy functionality for the ServiceNow Incident form. The implementation manages client-side behavior, validation, field visibility, and mandatory-field control while maintaining a structured and reusable configuration.